

软件测试与维护大作业实验报告

基于 Online Boutique 微服务 KPI 的 USAD 异常检测复现

学生：李响 陈祖名学号：2311467 2313163

一、实验目标

本大作业独立围绕微服务 KPI 异常检测设计实验，并在第三档优化阶段扩展到 Online Boutique 复杂开源微服务系统。实验复用课程中使用过的工具链，包括 Docker、Minikube、Kubernetes、Prometheus、Grafana、ChaosMesh、JMeter 和 Selenium，但实验流程、数据处理、算法复现和结果分析均在大作业项目中重新组织完成。

论文选择 KDD 2020 的 USAD: UnSupervised Anomaly Detection on Multivariate Time Series 作为主复现论文。本实验目标是将 Online Boutique 与可控示例数据中的多变量 KPI 转化为滑动窗口时间序列，训练异常检测模型识别故障注入后的异常窗口，并通过异常分数、重构误差、Precision、Recall 和 F1 评估复现效果。

二、论文方法概述

USAD 面向 IT 系统监控场景中的多变量时间序列异常检测。其核心思想是只使用正常阶段数据训练自编码器，使模型学习正常 KPI 之间的时序关系和变量相关性；当故障发生时，异常窗口难以被模型准确重构，重构误差会升高。

- 输入：多变量 KPI 时间序列，例如响应时延、错误率、CPU、内存、Pod 重启次数。
- 窗口化：将连续监控数据切分为固定长度滑动窗口。
- 模型：共享编码器与两个解码器构成双重重构结构。
- 评分：使用两个解码器的重构误差加权得到异常分数。
- 判定：以正常训练集异常分数高分位数作为阈值，超过阈值即为异常。

三、独立实验设计

Independent large homework experiment workflow



图 1 大作业独立实验流程

实验设计包括三条闭环：先使用可控示例 KPI 数据完成 USAD 基础复现，再在 Online Boutique 中采集真实 KPI，最后使用 ChaosMesh PodChaos 进行真实故障注入并同步采集 KPI。其中，早期真实 KPI 采集使用 kubectl scale 将 productcatalogservice 缩放到 0 个副本制造可重复故障窗口；补强实验已改为执行 ChaosMesh PodChaos，从而形成故障注入、数据采集、算法检测的同一链路证据。算法部分不依赖已有实验报告结论，而是由本项目脚本重新完成数据生成、窗口化、模型训练、异常检测和结果统计。

阶段	本大作业独立完成内容	对应文件
数据准备	生成可控示例 KPI，并采集 Online Boutique 真实 KPI 与 Prometheus 探针指标	src/generate_sample_data.py, src/collect_online_boutique_metrics.py
故障注入	定义 Pod Kill 与网络延迟两类故障	scripts/chaos-pod-kill.yaml, scripts/chaos-network-delay.yaml
ChaosMesh 闭环	执行 PodChaos 同时采集 120 个真实 KPI 采样点	src/collect_online_boutique_chaosmesh_metrics.py
算法复现	实现轻量 USAD 双解码器模型	src/usad_numpy.py
检测评估	计算异常分数、阈值、Precision、Recall、F1	src/run_usad.py
结果输出	生成 CSV、图表和实验摘要	outputs/

四、第三档目标优化设计

根据大作业评分标准，第三档需要在更复杂的开源微服务系统基础上完成部署、监控和维护，并完成 1-2 个微服务开发。因此，本项目在 USAD 复现实验基础上进行了第三档目标优化：已将基础系统扩展到 Online Boutique，并新增自研 anomaly-detector 微服务与 online-boutique-probe-exporter

微服务，前者将 USAD 异常检测从脚本封装为可部署的 Kubernetes 服务，后者负责将 Online Boutique 应用层请求数、响应时间和错误数暴露为 Prometheus 指标。

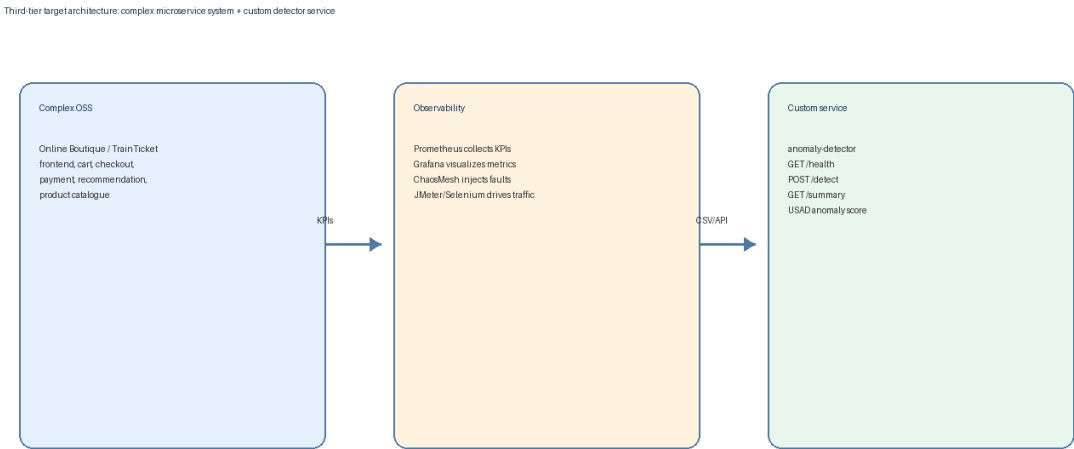


图 2 第三档目标架构：复杂微服务系统与自研异常检测服务

第三档要求	优化实现	项目文件
更复杂开源微服务系统	已部署 Online Boutique，Pod 均为 Running	docs/online_boutique_deployment_evidence.md
微服务开发	新增 anomaly-detector 异常检测服务	services/anomaly-detector/app.py
微服务开发	新增 online-boutique-probe-exporter 应用性能指标采集服务	services/online-boutique-probe-exporter/app.py
容器化	两个自研服务均提供 Dockerfile 构建服务镜像	services/*/Dockerfile
Kubernetes 部署	提供 Deployment、Service、健康检查和 Prometheus 抓取注解	k8s/
与运维流程集成	通过 /detect 调用 USAD，并通过 probe-exporter 暴露应用层指标	services/

本机部署记录显示 Online Boutique 所有核心 Pod 均已 Ready，并通过 port-forward 访问前端页面返回 HTTP 200。同时，anomaly-detector 已部署到 monitoring 命名空间，/health 返回正常，/detect 能实际运行 USAD 并返回 F1 等检测摘要。online-boutique-probe-exporter 也已接入 Prometheus，使 Grafana 能展示 Online Boutique 的请求数、响应时间和错误数等应用层性能指标。Prometheus 与 Grafana 已接入 Online Boutique 的 Kubernetes 状态指标，并补充了仪表盘截图作为运维监控证据。

五、实验环境与运行命令

实验项目位于 `large_homework/final_project`。模型实现使用 Python，主要依赖 `numpy`、`pandas`、`Pillow` 和 `python-docx`。若本机 Python 未安装依赖，可执行 ``pip install -r requirements.txt``。

```
python src/generate_sample_data.py --out data/sample_kpi_metrics.csv
python src/run_usad.py --input data/sample_kpi_metrics.csv --out outputs
```

若使用真实 Prometheus 数据，先通过 Minikube 暴露 Prometheus 服务，再执行导出脚本。导出后可根据 `kubectl scale` 或 `ChaosMesh` 注入时间段手动补充 `label` 字段。

```
python src/export_prometheus.py --prometheus http://127.0.0.1:9090 --start 2026-06-01T10:00:00 --end 2026-06-01T11:00:00 --step 30 --out data/prometheus_online_boutique_metrics.csv
```

Selenium 与 JMeter 测试证据

为满足大作业中对 Selenium 与 JMeter 的明确要求，本项目新增独立测试目录 `tests/selenium` 与 `tests/jmeter`。Selenium 脚本模拟用户打开 Online Boutique 首页、进入商品详情页并访问购物车页面，记录页面加载与交互耗时，并保存首页、商品页、购物车页截图。JMeter 测试计划使用 10 个并发线程循环访问首页、商品页和购物车页，共生成 240 个 HTTP 样本。实际运行结果显示 240 个样本全部成功，错误率 0.00%，平均响应时间 15.83 ms，最大响应时间 81 ms。本机浏览器兼容性先以 Chrome headless 作为代表浏览器完成，随后补充 Edge 浏览器复测；Firefox 因本机未安装，保留为后续可扩展项。

测试工具	实际产物	关键结果
Selenium	tests/selenium/online_boutique_ui_test.py, outputs_selenium/	首页、商品页、购物车页测试均 passed
JMeter	tests/jmeter/online_boutique_load_test.jmx, outputs_jmeter/	240 samples, error rate 0.00%, avg 15.83 ms

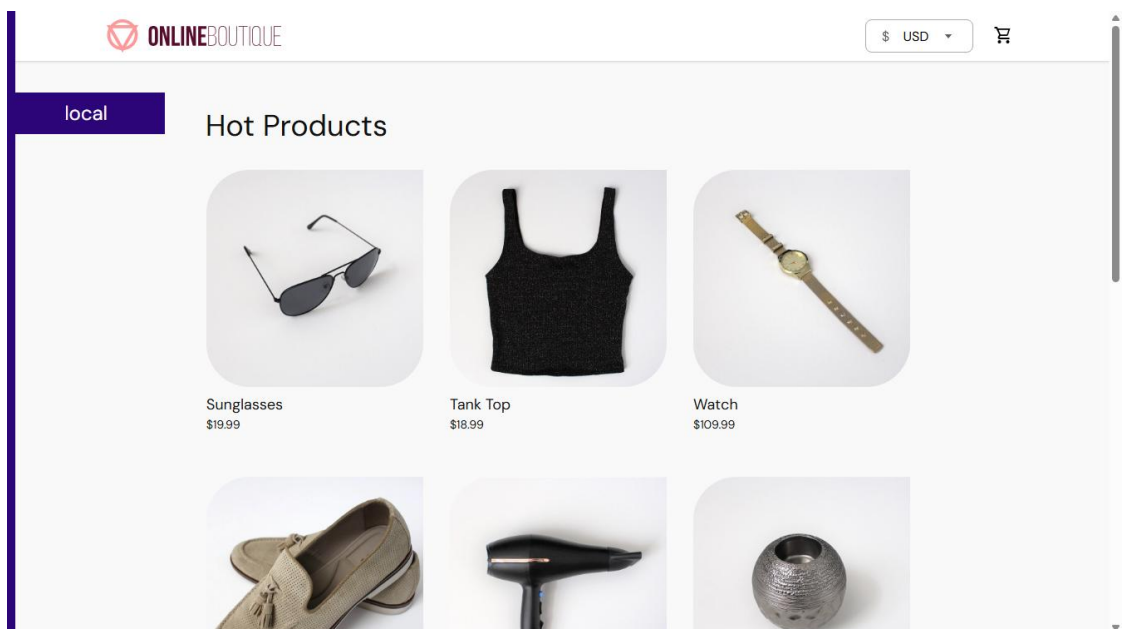


图 3 Selenium 打开 Online Boutique 首页截图

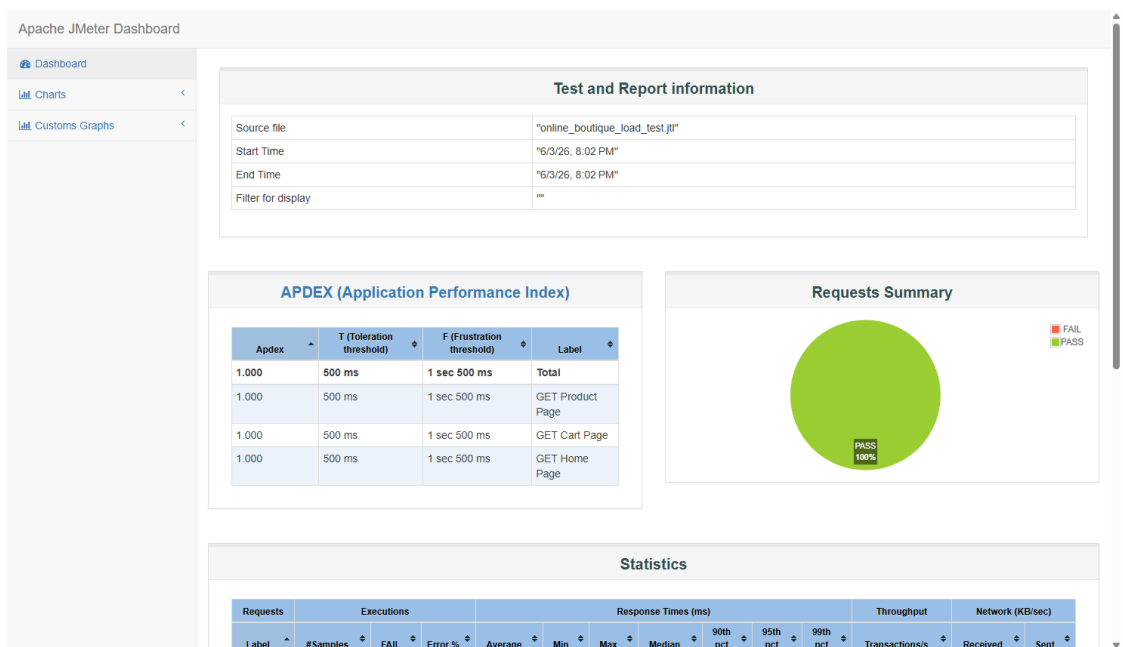


图 4 JMeter HTML 测试报告截图

Selenium 多浏览器与 JMeter 并发矩阵补强

为补充浏览器兼容性验证，本项目扩展 Selenium 脚本，使同一条用户路径可以在 Chrome、Edge 和 Firefox 上分别运行，并输出每个浏览器的页面加载耗时。

浏览器	状态	首页(ms)	商品页(ms)	购物车(ms)
edge	passed	1220.02	197.04	83.95

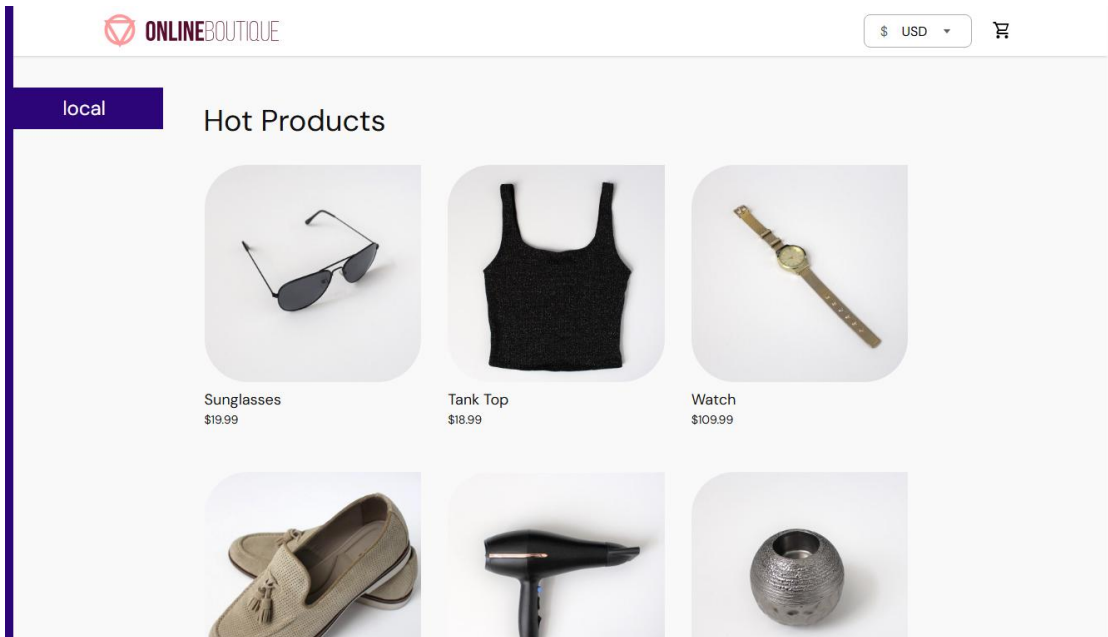


图 5 Edge 浏览器打开 Online Boutique 首页截图

为观察不同并发压力下的性能变化，本项目新增 **JMeter 10/30/50** 并发矩阵测试，并汇总错误率、平均响应时间、**P95** 和最大响应时间。

并发线程	样本数	错误率	平均响应(ms)	P95(ms)	最大响应(ms)
10	240	0.00%	40.20	94.00	164.00
30	720	0.00%	180.22	354.00	977.00
50	1200	0.00%	404.30	902.00	1344.00

JMeter 10/30/50 Concurrency Summary

Threads	Samples	Error	Avg ms	P95 ms	Max ms
10	240	0.00%	40.20	94.00	164.00
30	720	0.00%	180.22	354.00	977.00
50	1200	0.00%	404.30	902.00	1344.00

图 6 JMeter 10/30/50 并发对比摘要

六、故障注入方案

本实验准备了两类故障场景。第一类是 catalogue 服务 Pod Kill，用于模拟服务实例异常退出和重启；第二类是 front-end 网络延迟，用于模拟用户请求链路变慢。两类故障都会在 KPI 中体现为响应时延上升、错误率增加、资源使用变化或 Pod 重启计数变化。

除原有 ChaosMesh YAML 示例外，本项目已实际安装 ChaosMesh，并针对 Online Boutique 新增 scripts/chaos-online-boutique-productcatalog-podkill.yaml。执行 kubectl apply 后，PodChaos 状态中 Selected=True、AllInjected=True，事件显示 Successfully apply chaos，原 productcatalogservice Pod 被杀掉后由 Deployment 自动重建，新 Pod 恢复为 1/1 Running。

故障类型	注入对象	预期 KPI 表现	配置文件
Pod Kill	catalogue 服务	Pod 重启次数上升，相关 CPU/内存和前端时延波动	chaos-pod-kill.yaml
Network Delay	front-end 服务	响应时延上升，错误率和吞吐可能波动	chaos-network-delay.yaml
Pod Kill	Online Boutique productcatalogservice	Pod 被杀后自动重建，前端探针延迟可能波动	chaos-online-boutique-productcatalog-podkill.yaml

七、数据与模型实现

示例数据包含 720 个采样点和 9 个 KPI 字段，采样间隔按 30 秒模拟。正常阶段位于序列前部，后续插入两个故障窗口。真实实验中可直接替换为 Prometheus 导出的 CSV，只需保持 timestamp、KPI 列和可选 label 列。

USAD 复现版本使用纯 NumPy 实现，保留论文中的共享编码器和两个解码器结构。模型在正常窗口上训练，检测阶段对全量窗口计算异常分数。为便于课程环境运行，该实现不依赖 PyTorch，降低了复现和提交检查的环境成本。

八、实验结果

指标	结果
输入行数	720
KPI 数量	9
窗口大小	12
训练窗口数	240
训练轮数	180
阈值	0.677151

Precision	0.5889
Recall	1.0000
F1	0.7413

从本次运行结果看，USAD 对两个故障窗口均产生明显高于阈值的异常分数，召回率达到 1.0，说明故障窗口被完整捕获。Precision 为 0.5889，表示仍存在一定误报。误报主要来自正常样本数量较少、阈值估计偏保守以及示例数据扰动较强。在真实环境中，可通过延长正常阶段采集时间、调节阈值分位数和引入业务低谷/高峰样本来降低误报。

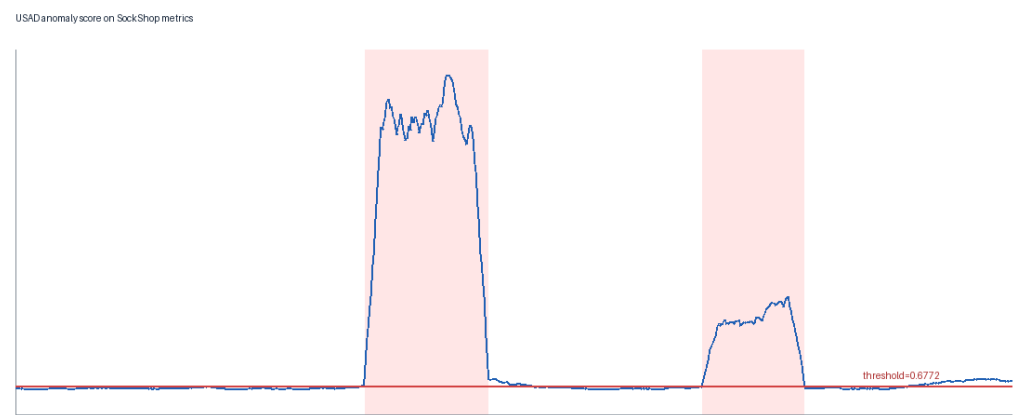


图 7 USAD 示例数据异常分数曲线

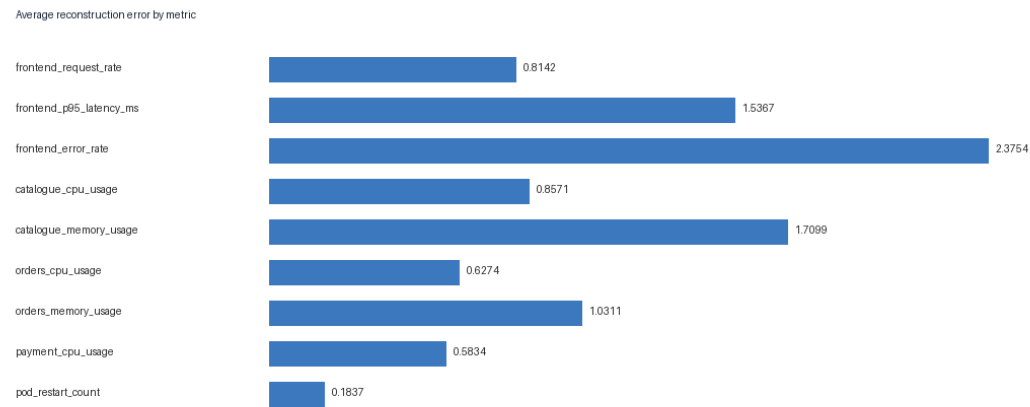


图 8 示例数据各 KPI 平均重构误差

九、Online Boutique 真实数据闭环

在第三档优化阶段，本项目进一步采集 Online Boutique 的真实运行数据，并执行实际故障注入。由于当前集群未安装 metrics-server，且 Online Boutique 前端未暴露 /metrics，本实验从前端 HTTP 实际访问结果和 Kubernetes API 中采集 KPI，包括响应时间、状态码、成功/失败标记、响应字节数、Pod Running/NotReady 数、总重启次数以及关键 Deployment Ready Ratio。

故障注入通过将 productcatalogservice 缩放到 0 个副本实现，持续 30 个采样点后恢复到 1 个副本。这会直接影响商品页面和商品目录相关访问，并在 frontend_status_code、frontend_latency_ms、frontend_success 以及 productcatalogservice_ready_ratio 等指标中体现。该阶段采用 kubectl scale 是为了获得边界清晰、便于标注的真实 KPI 数据；后续补强实验已改用 ChaosMesh PodChaos 同步采集 KPI，用于证明故障注入工具链和算法检测链路均已打通。

真实数据指标	结果
输入文件	data/online_boutique_real_metrics.csv
样本数	120
KPI 数量	15
窗口大小	6
Precision	0.6071
Recall	0.9714
F1	0.7473
Top 异常指标	frontend_status_code, frontend_latency_ms, productcatalogservice_ready_ratio

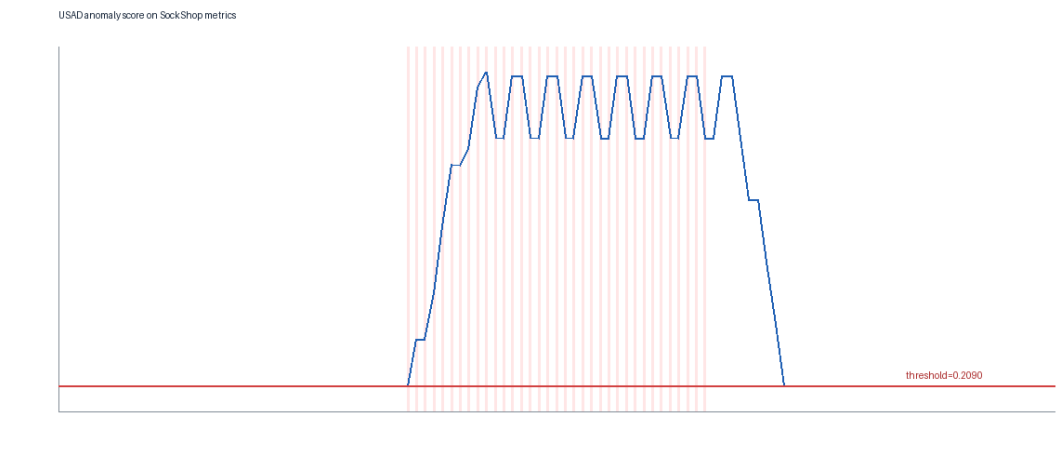


图 9 Online Boutique 真实 KPI 的 USAD 异常分数

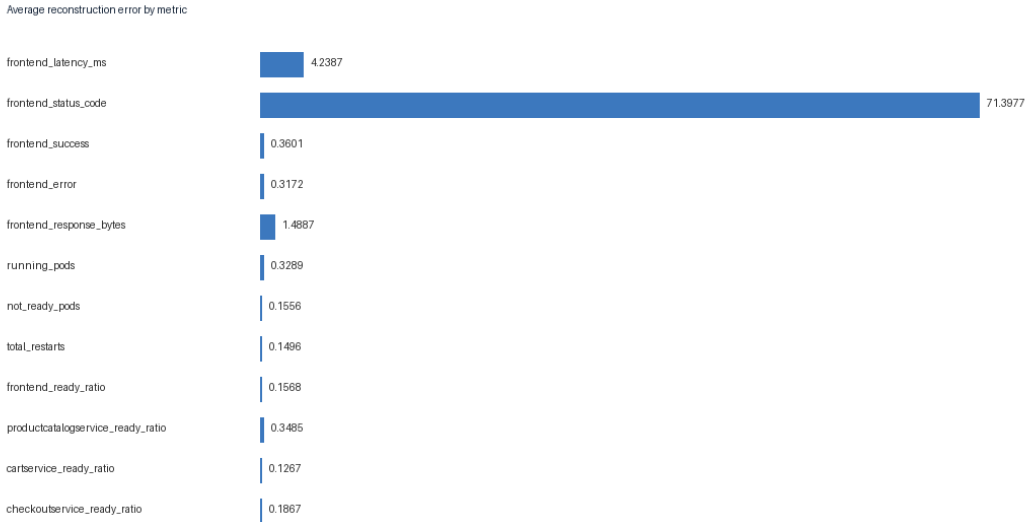


图 10 Online Boutique 真实 KPI 的重构误差

随后将真实数据重新打入 anomaly-detector 镜像，使用 Kubernetes 中运行的 anomaly-detector 服务调用 /detect 接口。接口返回 status=ok，并给出 Precision=0.6071、Recall=0.9714、F1=0.7473，说明自研微服务已经能够在集群内完成真实 Online Boutique 数据的 USAD 检测。

ChaosMesh 注入与 KPI 检测闭环

为避免只用 kubectl scale 作为故障来源，本项目新增 ChaosMesh KPI 闭环实验：执行 PodChaos 的同时采集 Online Boutique KPI，并将采集结果直接输入 USAD，形成“故障注入、数据采集、异常检测”的同一链路证据。

ChaosMesh KPI 指标	结果
输入文件	D:\HuaweiMoveData\Users\13976\Documents\New project_teammate_project_analysis\final_project\data\online_boutique_chaosmesh_metrics.csv
样本数	120
KPI 数量	16
窗口大小	6
Precision	0.4643
Recall	0.7429
F1	0.5714

ChaosMesh KPI Collection Evidence

Item	Value
PodChaos status	Selected=True, AllInjected=True
Fault window	samples 45-74
Rows / metrics	120 / 16
Precision / Recall	0.4643 / 0.7429
F1	0.5714
Top signal	frontend_latency_ms

图 11 ChaosMesh PodChaos 注入状态摘要

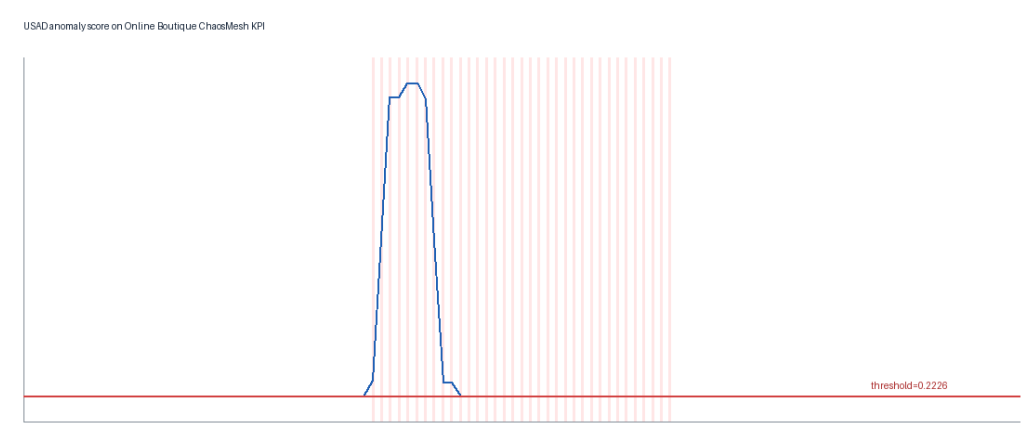


图 12 ChaosMesh KPI 的 USAD 异常分数

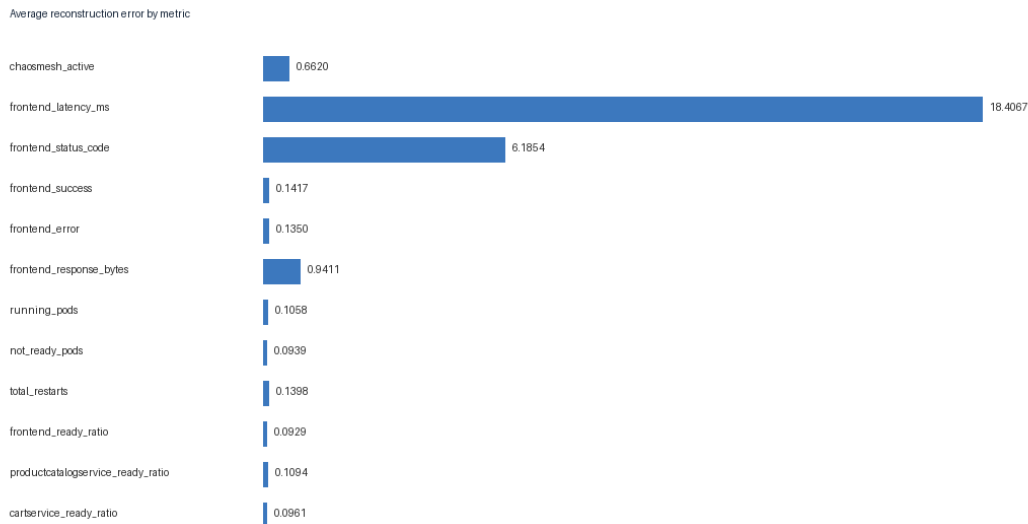


图 13 ChaosMesh KPI 的重构误差

Prometheus/Grafana 监控证据

为补强第三档所需的运维监控展示，本项目在 `monitoring` 命名空间部署 Prometheus、kube-state-metrics 与 Grafana。Prometheus 采用轻量配置采集 kube-state-metrics 与服务端点指标，避免在单节点 Minikube 中通过 apiserver proxy 抓取 cAdvisor 带来额外压力。同时新增 `online-boutique-probe-exporter`，使 Prometheus 能够直接采集应用层请求数、响应时间、错误数、状态码和响应字节数。Grafana 仪表盘展示 Online Boutique Ready Pod 数、容器重启次数、Frontend Probe Requests、Frontend Latency、Frontend Error Total、Pod 阶段分布和 Deployment Ready 副本数。当前 Ready Pod 查询结果为 12，说明 Online Boutique 全部核心服务已恢复可用；应用探针错误数为 0，最近响应时间约 100 ms。

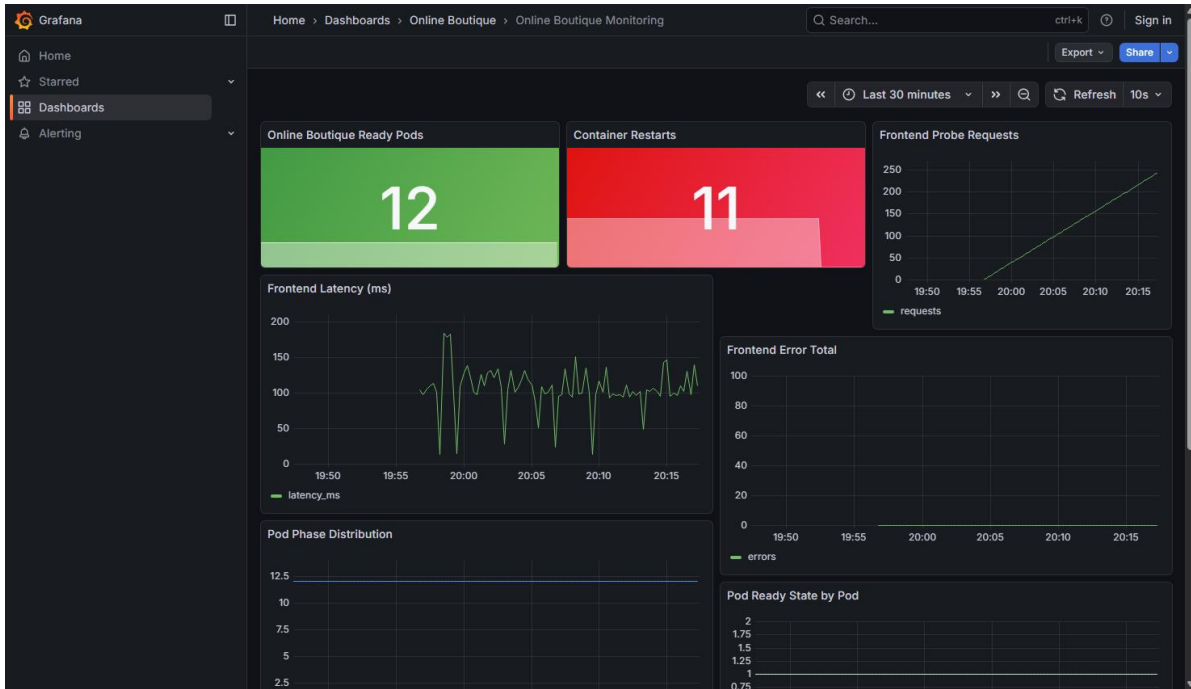


图 14 Online Boutique 的 Grafana 应用性能与运维监控仪表盘

十、异常解释

重构误差可以用于解释异常来源。本次 Online Boutique 与 ChaosMesh 实验中，frontend_latency_ms、frontend_status_code、frontend_response_bytes 等指标位于重构误差或异常贡献排名前列，符合 productcatalogservice Pod 被杀后对商品页面访问和前端体验产生影响的预期。因此，USAD 不仅可以给出窗口级异常判断，还能通过各 KPI 的重构误差为后续故障定位提供候选指标。

十一、实验不足与改进

- 当前示例数据规模较小，阈值估计容易受正常波动影响。
- 纯 NumPy 版本保留了 USAD 核心结构，但未完全复现论文中的深度学习训练细节。
- 真实环境应采集更长时间的 Prometheus 数据，并进一步扩大故障类型，例如 CPU 压力、网络延迟和多服务级联故障。
- 当前已完成真实 Online Boutique KPI 检测，并补充 Prometheus/Grafana 仪表盘截图；后续可将异常分数继续写回监控系统形成告警闭环。

十二、结论

本大作业完成了基于微服务监控指标的 USAD 异常检测复现实验。实验从 KPI 数据构造、故障注入方案、模型训练、异常检测、结果评价到报告输出均在 `final_project` 中实现。为对齐第三档评分标准，项目进一步补充了 `anomaly-detector` 自研异常检测微服务、`online-boutique-probe-exporter` 应用性能指标采集微服务、`Dockerfile`、`Kubernetes` 部署清单和复杂系统迁移方案。结果表明，基于多变量时间序列重构误差的无监督异常检测方法能够有效捕获微服务故障注入后的异常窗口，并可封装为智能运维微服务接入更复杂的微服务系统。